

DATABASE MANAGEMENT SYSTEMS

Complete Unit 1 & Unit 2 Notes

B.Tech 3rd Year — AKTU

What this PDF contains

- ◆ **Unit 1: DBMS fundamentals, architecture, data independence, keys, ER/EER modeling and ER-to-relational mapping**
- ◆ **Unit 2: Relational model, integrity constraints, relational algebra and calculus, SQL, views, triggers and authorization**
- ◆ **Visual learning: Mind maps, architecture diagrams, ER mapping, operator map, SQL flow and join visualization**
- ◆ **Exam support: Definitions, comparisons, solved examples, important questions, practice problems and rapid-revision sheets**

Designed for fast concept building, university examination preparation and last-minute revision.

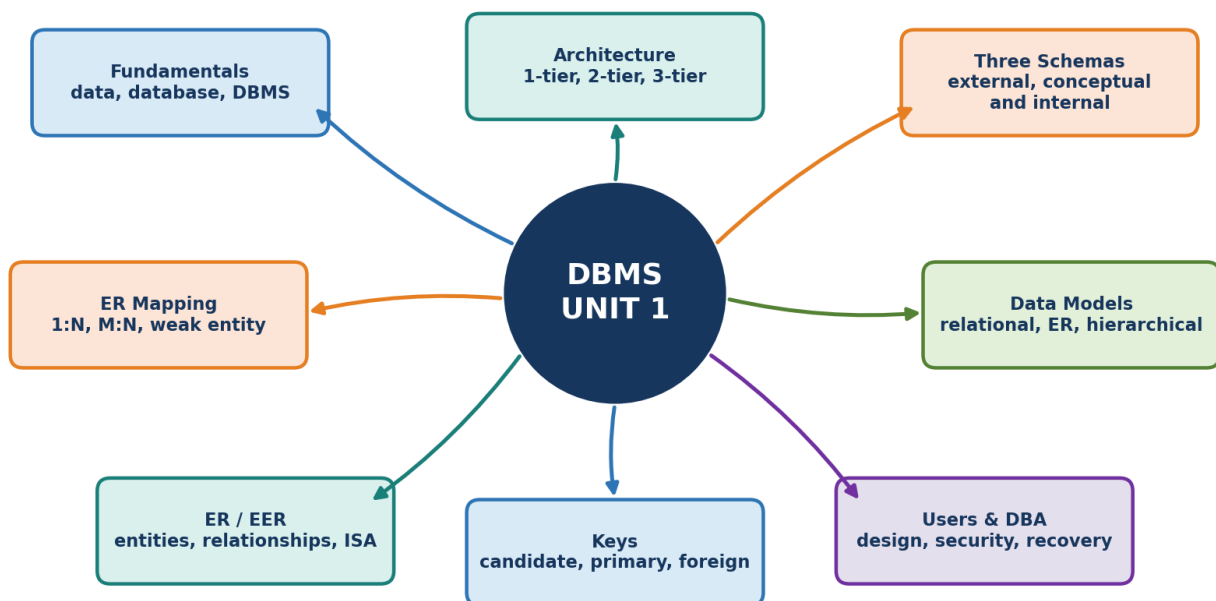
DBMS Unit 1 Notes

B.Tech 3rd Year — AKTU (Exam-Oriented)

Scope: These notes cover the topics commonly prescribed in AKTU DBMS Unit 1: introduction to database systems, DBMS architecture, data models, schemas and instances, data independence, database languages, users/DBA, keys and constraints, ER/EER modeling, and ER-to-relational mapping.

Unit 1 Visual Mind Map

Unit 1 Mind Map — Database Fundamentals and ER Modeling



Unit 1 mind map showing all major DBMS fundamentals and ER modeling topics

Use the mind map as a study route: begin with fundamentals and architecture, then learn schemas and keys, and finally practice ER/EER design and relational mapping.

1. Database and DBMS

1.1 Data

Data consists of raw facts that can be recorded and have an implicit meaning.

Examples: student roll number, name, marks, account balance, product price.

1.2 Information

Information is processed or organized data that is meaningful and useful for decision-making.

Example: "The average marks of the class are 72" is information obtained by processing students' marks.

1.3 Database

A **database** is an organized collection of logically related data designed to meet the information needs of an organization.

Example: A university database may contain data about students, courses, teachers, departments, and examinations.

1.4 Database Management System

A **Database Management System (DBMS)** is software that enables users to define, create, store, retrieve, update, and control access to a database.

Examples: MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server and SQLite.

1.5 Database System

A **database system** consists of:

1. Database
 2. DBMS software
 3. Application programs
 4. Users
 5. Hardware and operating system
- Users Application/SQL DBMS Database

1.6 Main Characteristics of a DBMS

- **Self-describing:** Stores metadata in a system catalog/data dictionary.
- **Program-data independence:** Data structure can change without rewriting every application.
- **Multiple views:** Different users can see different parts of the database.
- **Data sharing:** Many users and applications can access common data.
- **Transaction processing:** A group of operations can be treated as one logical unit.
- **Controlled redundancy:** Unnecessary duplication is minimized.
- **Integrity enforcement:** Rules maintain valid and consistent data.
- **Security:** Unauthorized access is restricted.
- **Backup and recovery:** Data can be restored after failure.
- **Concurrency control:** Simultaneous operations are managed safely.

2. File Processing System vs DBMS

Before DBMSs, organizations stored data in separate application files.

Basis	File Processing System	DBMS
Data storage	Separate files	Integrated database
Redundancy	High	Controlled
Consistency	Difficult to maintain	Better consistency
Data sharing	Limited	Easy and controlled
Program-data dependence	High	Low
Security	Application-specific	Centralized authorization
Integrity constraints	Written in each program	Defined in DBMS
Concurrent access	Difficult	Concurrency control provided
Backup/recovery	Manual or limited	DBMS facilities available
Querying	New program often required	SQL/ad hoc queries
Cost	Lower for very small tasks	Higher setup and administration cost

Problems with Traditional File Systems

1. **Data redundancy:** The same data is stored in many files.

2. **Data inconsistency:** Different copies may contain different values.
3. **Data isolation:** Data is scattered across files and formats.
4. **Difficult data access:** A new program may be needed for every request.
5. **Integrity problems:** Business rules are hard-coded repeatedly.
6. **Atomicity problems:** A failure may leave an operation partially completed.
7. **Concurrent-access anomalies:** Simultaneous updates can conflict.
8. **Security problems:** Uniform access control is difficult.

Disadvantages of a DBMS

- High initial cost
- Complex software and administration
- Requires trained staff
- Additional hardware/storage overhead
- Failure of a centralized DBMS can affect many users
- Not economical for every small, single-user application

3. DBMS Environment and Users

3.1 Components of a DBMS Environment

- **Hardware:** Servers, storage devices and client systems
- **Software:** DBMS, operating system and applications
- **Data:** Operational data and metadata
- **Procedures:** Rules for design, use, backup and security
- **People:** DBA, designers, developers and end users
- **Access language:** Usually SQL

3.2 Types of Database Users

1. **Database Administrator (DBA):** Controls the overall database system.
2. **Database designers:** Identify data, relationships and constraints.
3. **System analysts/application programmers:** Develop database applications.
4. **End users:**
 - **Naive/parametric users:** Use predefined forms, such as bank clerks.
 - **Casual users:** Access the database occasionally using queries.
 - **Sophisticated users:** Use analysis tools or write complex queries.
 - **Standalone users:** Maintain personal databases using packaged software.
5. **Specialized users:** Build applications such as AI, CAD, GIS or scientific systems.

3.3 Responsibilities of a DBA

- Define and modify database schema
- Choose storage structures and access methods
- Create users and grant/revoke privileges
- Enforce integrity constraints
- Monitor performance and tune the database
- Manage concurrency and availability

- Plan backup and recovery
- Maintain the data dictionary
- Perform capacity planning
- Handle security and auditing

Exam tip: For “Role of DBA,” write at least six responsibilities with one-line explanations.

4. Data Abstraction

A DBMS hides unnecessary storage details and presents users with appropriate views.

4.1 Levels of Data Abstraction

1. **Physical/Internal level:** Describes how records are stored—files, blocks, indexes and access paths.
2. **Logical/Conceptual level:** Describes what data is stored and the relationships and constraints.
3. **View/External level:** Describes only the portion of the database needed by a particular user.

Highest abstraction

External/View Level User-specific views

Conceptual/Logical Level Complete logical database structure

Internal/Physical Level Storage files, blocks and indexes

Lowest abstraction

Example:

- Physical: Student records are stored in disk blocks using a B+ tree index.
- Logical: `STUDENT(RollNo, Name, Branch, CGPA)`.
- View: A teacher sees RollNo, Name and marks but not fee details.

5. Schema, Instance and Database State

5.1 Schema

A **schema** is the overall logical design or structure of a database. It changes infrequently and is also called the **intension** of the database.

Example:

`STUDENT(RollNo, Name, Branch, CGPA)`

5.2 Instance

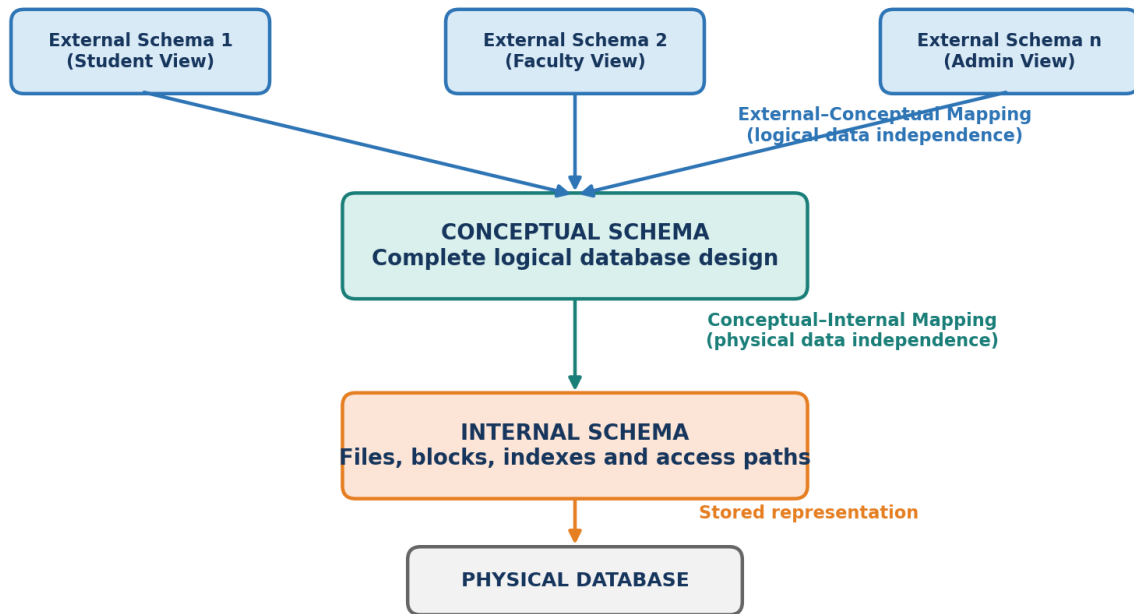
A **database instance** is the actual data stored at a particular moment. It changes whenever data is inserted, deleted or updated and is also called the **extension** or database state.

Schema vs Instance

Schema	Instance
Blueprint/design of database	Data present at a specific time
Changes rarely	Changes frequently
Defined using DDL	Modified using DML
Also called intension	Also called extension/state

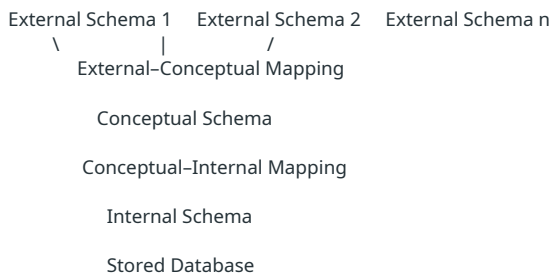
5.3 Three-Schema Architecture (ANSI/SPARC Architecture)

Three-Schema Architecture and Data Independence



Three-schema architecture showing external, conceptual and internal levels with data independence

The architecture separates user applications from physical storage.



- **External schema:** A particular user's view.
- **Conceptual schema:** The complete logical structure of the database.
- **Internal schema:** Physical storage representation.
- **Mappings:** Translate requests and results between levels.

Advantages

- Supports multiple views
- Improves security
- Hides storage details
- Provides logical and physical data independence
- Simplifies database evolution

6. Data Independence

Data independence is the ability to change the schema at one level without requiring changes at the next higher level.

6.1 Physical Data Independence

The ability to modify the internal schema without changing the conceptual schema or application programs.

Examples:

- Changing file organization
- Adding/removing an index
- Moving data to another storage device
- Changing compression technique

6.2 Logical Data Independence

The ability to modify the conceptual schema without changing external schemas or application programs.

Examples:

- Adding a new attribute
- Splitting one relation into two
- adding a new entity or relationship
- Changing a constraint while preserving user views

Comparison

Physical Data Independence	Logical Data Independence
Change at internal level	Change at conceptual level
Higher levels remain unaffected	External views/programs remain unaffected
Easier to achieve	More difficult to achieve
Example: add an index	Example: add a new column/entity

Important: Logical data independence is harder because application programs depend more heavily on the logical structure of data.

7. Data Models

A **data model** is a collection of concepts used to describe data, relationships, constraints and operations.

7.1 Categories of Data Models

A. High-Level/Conceptual Data Models

Close to the way users understand data.

- Entity-Relationship (ER) model
- Object-based models

B. Logical/Representational Data Models

Describe data logically while hiding many storage details.

- Relational model
- Hierarchical model
- Network model
- Object-relational model

C. Physical Data Models

Describe how data is stored using record formats, blocks, pointers and access paths.

7.2 Important Models

Hierarchical Model

- Organizes data as a tree.
- Each child normally has one parent.

- Naturally supports one-to-many relationships.
- Navigation starts from the root.

University
 Department
 Teacher
 Student
 Library

Network Model

- Organizes records as a graph.
- A child can have multiple parents.
- Supports many-to-many relationships.
- Uses pointers/links and is navigational.

Relational Model

- Represents data in tables called relations.
- Rows are tuples and columns are attributes.
- Relationships are represented using common attributes/foreign keys.
- SQL is commonly used.

Entity-Relationship Model

- Represents the real world using entities, attributes and relationships.
- Mainly used for conceptual database design.

Object-Oriented Model

- Stores data as objects with identity, state and behavior.
- Supports encapsulation, inheritance and complex data.

Comparison of Major Models

Model	Structure	Major strength	Limitation
Hierarchical	Tree	Simple for one-to-many data	Poor many-to-many support
Network	Graph	Flexible relationships	Complex navigation
Relational	Tables	Simplicity and powerful queries	Complex objects need mapping
ER	Diagram/concepts	Excellent for design	Not normally a storage model
Object-oriented	Objects/classes	Handles complex objects	Less universal standardization

8. Database Languages and Interfaces

8.1 Data Definition Language (DDL)

Used to define and modify database structures.

Common commands: CREATE, ALTER, DROP, TRUNCATE, RENAME.

```
CREATE TABLE Student (
  RollNo INT PRIMARY KEY,
  Name VARCHAR(50),
  Branch VARCHAR(10)
);
```

8.2 Data Manipulation Language (DML)

Used to retrieve and modify data.

Common commands: SELECT, INSERT, UPDATE, DELETE.

```
SELECT Name FROM Student WHERE Branch = 'CSE';
```

DML may be:

- **Procedural:** Specifies what data is required and how to obtain it.
- **Non-procedural/declarative:** Specifies what data is required, not the retrieval procedure. SQL is largely declarative.

8.3 Data Control Language (DCL)

Controls authorization and privileges.

Commands: GRANT, REVOKE.

8.4 Transaction Control Language (TCL)

Manages transactions.

Commands: COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION.

8.5 View Definition Language (VDL)

Conceptually specifies external views. In SQL, views are created through CREATE VIEW.

8.6 Storage Definition Language (SDL)

Conceptually specifies the internal schema. Modern DBMSs usually provide storage options through DDL or administrative commands.

8.7 Database Interfaces

- Command-line SQL interface
- Menu-based interface
- Forms-based interface
- Graphical user interface
- Natural-language interface
- Web/mobile application interface
- APIs such as JDBC and ODBC

9. DBMS Structure

A simplified DBMS contains a **query processor** and a **storage manager**.

Users / Application Programs

Query Processor

DDL Interpreter
DML Compiler/Parser
Query Optimizer
Query Execution Engine

Storage Manager

Authorization &
Integrity Manager
Transaction Manager
File Manager
Buffer Manager

Data Files | Indexes | Catalog | Log

9.1 Query Processor Components

- **DDL interpreter:** Processes schema definitions and records metadata.
- **DML compiler/parser:** Translates queries into an internal form.
- **Query optimizer:** Selects an efficient execution plan.

- **Execution engine:** Executes the selected plan.

9.2 Storage Manager Components

- **Authorization and integrity manager:** Checks permissions and constraints.
- **Transaction manager:** Maintains consistency despite failures/concurrency.
- **File manager:** Manages allocation and organization of disk files.
- **Buffer manager:** Moves data between disk and main memory.

9.3 Stored Information

- **Data files:** Actual records
- **Indexes:** Faster paths to records
- **Data dictionary/system catalog:** Metadata
- **Log:** History needed for recovery

9.4 Data Dictionary

A **data dictionary** is a centralized repository of metadata, such as:

- Table and column names
- Data types and sizes
- Constraints
- Relationships
- Users and privileges
- Indexes and views
- Storage statistics

10. DBMS Architectures

10.1 Centralized Architecture

The database and DBMS are located on one central computer. Users connect through terminals or client systems.

Advantages: Simple administration and control.

Limitations: Central bottleneck and single point of failure.

10.2 One-Tier Architecture

The user directly accesses the DBMS, usually on the same system. Common in local development.

User/Application + DBMS + Database

10.3 Two-Tier Architecture

A client application directly communicates with a database server, usually through JDBC/ODBC.

Client Application Database Server

- Client handles interface and some business logic.
- Server handles query processing and data storage.

10.4 Three-Tier Architecture

An application server is placed between the client and database server.

Presentation Tier Application/Business Tier Database Tier

Advantages:

- Better security

- Greater scalability
- Easier maintenance
- Centralized business logic
- Suitable for web applications

Two-Tier vs Three-Tier

Two-Tier	Three-Tier
Client directly connects to DB	Client connects through application server
Business logic often on client	Business logic in middle tier
Suitable for smaller systems	Suitable for large/web systems
Harder to scale and secure	Better scalability and security

Entity–Relationship (ER) Model

11. Basic Concepts

11.1 Entity

An **entity** is a distinguishable real-world object with an independent existence.

Examples: a particular student, course, employee or department.

11.2 Entity Type

An **entity type** is a collection of entities having the same attributes.

Example: *STUDENT* is an entity type; "Student with RollNo 101" is an entity.

11.3 Entity Set

An **entity set** is the collection of all entities of an entity type present in the database at a particular time.

11.4 Attributes

An **attribute** describes a property of an entity or relationship.

Example: Student has RollNo, Name, DateOfBirth and Address.

Types of Attributes

1. **Simple/atomic attribute:** Cannot be divided meaningfully.

Example: Gender, Salary.

2. **Composite attribute:** Can be divided into components.

Example: Name FirstName, MiddleName, LastName.

3. **Single-valued attribute:** Has one value for each entity.

Example: DateOfBirth.

4. **Multi-valued attribute:** May have multiple values.

Example: PhoneNumbers, Skills.

5. **Stored attribute:** Its value is explicitly stored.

Example: DateOfBirth.

6. **Derived attribute:** Computed from other attributes.

Example: Age derived from DateOfBirth.

7. **Key attribute:** Uniquely identifies an entity.

Example: RollNo.

8. **Null/optional attribute:** May have no applicable or known value.

Example: MiddleName.

9. **Complex attribute:** Combination of composite and multivalued attributes.

11.5 ER Diagram Notation

Symbol	Meaning
Rectangle	Strong entity type
Double rectangle	Weak entity type
Diamond	Relationship type
Double diamond	Identifying relationship
Ellipse	Attribute
Double ellipse	Multivalued attribute
Dashed ellipse	Derived attribute
Underlined attribute	Key attribute
Double line	Total participation
Single line	Partial participation
Triangle (ISA)	Specialization/generalization

12. Keys

A **super key** is any set of one or more attributes that uniquely identifies a tuple/entity.

Consider:

STUDENT(RollNo, AadhaarNo, Email, Name, Branch)

12.1 Types of Keys

1. **Super key:** Any unique attribute set.

Examples: {RollNo}, {Email}, {RollNo, Name}.

2. **Candidate key:** A minimal super key—no proper subset is a super key.

Examples: {RollNo}, {AadhaarNo}, {Email}.

3. **Primary key:** Candidate key selected to identify records.

Example: RollNo.

4. **Alternate key:** Candidate keys not selected as the primary key.

Examples: AadhaarNo, Email.

5. **Composite/compound key:** Key containing more than one attribute.

Example: (StudentId, CourseId) in ENROLLMENT.

6. **Foreign key:** Attribute(s) in one relation that reference the primary/candidate key of another relation.

Example: DeptId in STUDENT references DEPARTMENT.

7. **Surrogate key:** Artificial system-generated key without business meaning.

Example: auto-incremented StudentId.

8. **Partial key/discriminator:** Attribute that uniquely identifies a weak entity within its owner entity.

Example: DependentName within an Employee's dependents.

Key Relationship

Super Keys

Candidate Keys (minimal super keys)

Primary Key (selected candidate key)

Alternate Keys (remaining candidate keys)

13. Relationships

A **relationship** is an association among entities. A **relationship type** defines similar associations, while a **relationship set** contains current relationship instances.

Example: STUDENT **enrolls in** COURSE.

13.1 Degree of Relationship

- **Unary/recursive:** Relationship among entities of the same type.

Example: EMPLOYEE supervises EMPLOYEE.

- **Binary:** Relationship between two entity types.

Example: STUDENT enrolls in COURSE.

- **Ternary:** Relationship among three entity types.

Example: SUPPLIER supplies PART to PROJECT.

- **n-ary:** Relationship among n entity types.

13.2 Role Names

Role names explain the function played by an entity, especially in recursive relationships.

Example: In EMPLOYEE supervises EMPLOYEE, roles are **Supervisor** and **Subordinate**.

13.3 Relationship Attributes

A relationship can have attributes.

Example: ENROLLS(Student, Course) may have **EnrollmentDate** and **Grade**.

14. Structural Constraints

14.1 Cardinality Ratio

Specifies the maximum number of relationship instances in which an entity may participate.

One-to-One (1:1)

One entity of A is associated with at most one entity of B and vice versa.

Example: PERSON — HAS — PASSPORT.

PERSON 1 1 PASSPORT

One-to-Many (1:N)

One A can be associated with many B entities, while each B is associated with at most one A.

Example: DEPARTMENT — employs — EMPLOYEE.

DEPARTMENT 1 N EMPLOYEE

Many-to-One (N:1)

Many A entities can be associated with one B entity.

Example: Many students belong to one department.

Many-to-Many (M:N)

Many A entities can be associated with many B entities.

Example: STUDENT — enrolls in — COURSE.

STUDENT M N COURSE

14.2 Participation Constraint

Specifies the minimum participation of an entity.

- **Total participation:** Every entity must participate in at least one relationship instance. Shown by a double line.
- **Partial participation:** Some entities may not participate. Shown by a single line.

Example: If every employee must belong to a department, EMPLOYEE has total participation in WORKS_FOR. If a department may temporarily have no employee, DEPARTMENT has partial participation.

14.3 Min-Max Notation

(min, max) gives the minimum and maximum participation.

- (0,1): optional, at most one
- (1,1): exactly one
- (0,N): optional, many allowed
- (1,N): at least one, many allowed

Do not confuse: Cardinality describes the **maximum** participation; total/partial participation describes the **minimum** participation.

15. Strong and Weak Entity Types

15.1 Strong Entity

A **strong entity type** has its own primary key and exists independently.

Example: EMPLOYEE(EmployeeId, Name).

15.2 Weak Entity

A **weak entity type**:

- Does not have a complete key of its own
- Depends on an owner/identifying entity for existence
- Has a partial key (discriminator)
- Has total participation in its identifying relationship
- Is identified by owner key + partial key

Example:

```
EMPLOYEE(EmployeeId, Name)
DEPENDENT(DependentName, DOB, Relation)
```

A dependent is identified by (EmployeeId, DependentName).

```
[EMPLOYEE] 1 <HAS> N [[DEPENDENT]]
```

Here, DEPENDENT is weak and HAS is the identifying relationship.

Strong vs Weak Entity

Strong Entity	Weak Entity
Has its own primary key	Has only a partial key
Independent existence	Existence depends on owner
Single rectangle	Double rectangle
Normal relationship	Identifying relationship shown by double diamond
Participation may be partial	Total in identifying relationship

16. Design Issues in ER Modeling

16.1 Entity Set vs Attribute

Use an **attribute** for a simple value without its own properties. Use an **entity** when it has independent existence, several properties or relationships.

Example: Treat Address as a composite attribute if only its text is needed; treat it as an entity if multiple people share addresses and address history must be stored.

16.2 Entity Set vs Relationship Set

If a concept associates entities, model it as a relationship. If the association itself needs many attributes, identity, or participation in other relationships, model it as an associative entity.

Example: ENROLLMENT can be modeled as an associative entity with StudentId, CourseId, Grade and Semester.

16.3 Binary vs Ternary Relationship

A ternary relationship cannot always be replaced by three binary relationships without losing meaning.

Example: "Supplier S supplies Part P to Project J" depends jointly on all three entities.

16.4 Placement of Relationship Attributes

Place an attribute where its value is uniquely determined.

Example: In EMPLOYEE-WORKS_ON-PROJECT, **Hours** belongs to the relationship because it depends on both employee and project.

16.5 Avoid Redundant Relationships

Do not add relationships that can always be derived from other relationships unless required for performance or historical meaning.

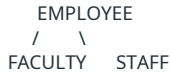
Enhanced ER (EER) Model

17. Specialization and Generalization

17.1 Specialization

A **top-down** process of dividing a superclass into subclasses based on distinguishing characteristics.

Example:



17.2 Generalization

A **bottom-up** process of combining similar lower-level entity types into a common superclass.

Example: CAR and TRUCK are generalized into VEHICLE.

Specialization vs Generalization

Specialization	Generalization
Top-down	Bottom-up
One superclass to subclasses	Multiple entity types to superclass
Finds differences	Finds common features
Example: Employee Faculty, Staff	Car, Truck Vehicle

17.3 Inheritance

A subclass inherits:

- Attributes of the superclass
- Relationships of the superclass
- Its own specific attributes and relationships

Example: FACULTY inherits EmployeeId and Name from EMPLOYEE and may add ResearchArea.

18. Constraints on Specialization

18.1 Disjointness Constraint

- **Disjoint specialization:** A superclass entity can belong to only one subclass. Marked **d**.
- **Overlapping specialization:** A superclass entity may belong to multiple subclasses. Marked **o**.

Example: An employee may be either salaried or hourly (disjoint), but a person may be both student and employee (overlapping).

18.2 Completeness Constraint

- **Total specialization:** Every superclass entity must belong to at least one subclass. Shown by a double line.
- **Partial specialization:** A superclass entity need not belong to a subclass. Shown by a single line.

Four Possible Combinations

1. Disjoint and total
2. Disjoint and partial
3. Overlapping and total

4. Overlapping and partial

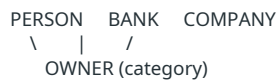
18.3 Attribute-Defined vs User-Defined Specialization

- **Attribute/predicate-defined:** Subclass membership is determined by an attribute condition.
- **User-defined:** Membership is assigned explicitly by a user.

19. Category/Union Type

A **category** is a subclass whose superclass is the union of two or more distinct entity types.

Example: An OWNER may be a PERSON, BANK or COMPANY.



Unlike ordinary specialization, the superclasses may have different entity types.

20. Aggregation

Aggregation is an abstraction in which a relationship set is treated as a higher-level entity so that it can participate in another relationship.

Example:

- EMPLOYEE works on PROJECT.
- MANAGER monitors the “employee works on project” association.

MANAGER monitors [EMPLOYEE works_on PROJECT]

Aggregation is useful when a relationship itself must be related to another entity.

ER-to-Relational Mapping

21. Mapping Rules

Rule 1: Strong Entity Type

Create one relation containing all simple attributes. Expand composite attributes into components. Choose the entity key as the primary key.

```
STUDENT(RollNo, FirstName, LastName, DOB)
PK: RollNo
```

Derived attributes are usually not stored.

Rule 2: Weak Entity Type

Create a relation containing:

- Weak entity's simple attributes
- Owner entity's primary key as a foreign key
- Primary key = owner key + partial key

```
DEPENDENT(EmployeeId, DependentName, DOB, Relation)
PK: (EmployeeId, DependentName)
FK: EmployeeId EMPLOYEE
```

Rule 3: Binary 1:1 Relationship

Place the primary key of one entity as a foreign key in the other, preferably on the side with total participation. Include relationship attributes there.

Example:

```
PERSON(PersonId, Name, PassportNo)
PASSPORT(PassportNo, IssueDate)
```

If PERSON has total participation, `PassportNo` can be placed in PERSON with `UNIQUE` and `NOT NULL` constraints.

Rule 4: Binary 1:N Relationship

Place the primary key of the "1" side as a foreign key in the relation on the "N" side. Put relationship attributes on the N side.

```
DEPARTMENT(DeptId, DeptName)
EMPLOYEE(EmpId, Name, DeptId)
FK: EMPLOYEE.DeptId DEPARTMENT.DeptId
```

Rule 5: Binary M:N Relationship

Create a new relation containing primary keys of both participating entities as foreign keys. Their combination normally becomes the primary key. Include relationship attributes.

```
STUDENT(StudentId, Name)
COURSE(CourseId, Title)
ENROLLMENT(StudentId, CourseId, Semester, Grade)
PK: (StudentId, CourseId, Semester)
```

Rule 6: Multivalued Attribute

Create a separate relation containing the owner key and the multivalued attribute. Their combination is the primary key.

```
STUDENT_PHONE(StudentId, PhoneNo)
PK: (StudentId, PhoneNo)
```

Rule 7: n-ary Relationship

Create a separate relation containing the primary keys of all participating entity types as foreign keys, along with relationship attributes. Select the key according to cardinality constraints.

SUPPLY(SupplierId, PartId, ProjectId, Quantity)

Rule 8: Recursive Relationship

- For recursive 1:N, use a self-referencing foreign key.
- For recursive M:N, create a separate relation with role-specific foreign keys.

EMPLOYEE(EmpId, Name, SupervisorId)
SupervisorId EMPLOYEE.EmpId

Rule 9: Specialization/Generalization

Common options are:

- One relation for superclass and one for each subclass.
- Relations only for subclasses, each including inherited attributes (best when specialization is total and disjoint).
- One relation with a type attribute (best for disjoint specialization).
- One relation with multiple Boolean type attributes (supports overlapping subclasses).

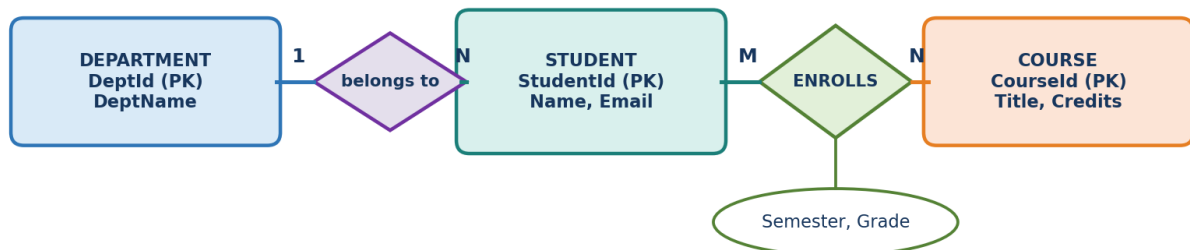
Most general method: superclass table + subclass tables.

EMPLOYEE(EmpId, Name, Salary)
FACULTY(EmpId, ResearchArea)
STAFF(EmpId, Skill)

FACULTY.EmpId and STAFF.EmpId are both primary keys and foreign keys referencing EMPLOYEE.

22. Worked ER Example: University Database

University ER Diagram and Relational Mapping



Mapped Relational Schema



Rule: 1:N → foreign key on N side | M:N → new relation containing both foreign keys

University ER diagram and its visual mapping into relational schemas

Requirements

- Each department has a unique DeptId and a name.
- Each student has a unique StudentId, name and email.
- Every student belongs to one department; a department has many students.
- Each course has a unique CourseId, title and credits.

- Students can enroll in many courses and courses can have many students.
- Enrollment records semester and grade.

Conceptual ER View

DEPARTMENT 1 N STUDENT

STUDENT M ENROLLMENT N COURSE

Semester Grade

Relational Schema

```
DEPARTMENT(  
  DeptId PK,  
  DeptName  
)
```

```
STUDENT(  
  StudentId PK,  
  Name,  
  Email UNIQUE,  
  DeptId FK DEPARTMENT.DeptId  
)
```

```
COURSE(  
  CourseId PK,  
  Title,  
  Credits  
)
```

```
ENROLLMENT(  
  StudentId FK STUDENT.StudentId,  
  CourseId FK COURSE.CourseId,  
  Semester,  
  Grade,  
  PK(StudentId, CourseId, Semester)  
)
```

Why This Mapping Is Correct

- The 1:N department–student relationship is implemented by **DeptId** on the N side.
- The M:N student–course relationship becomes ENROLLMENT.
- Relationship attributes Semester and Grade are stored in ENROLLMENT.

23. Integrity Constraints — Quick Introduction

An **integrity constraint** is a rule that keeps database data accurate and valid.

1. **Domain constraint:** Attribute values must come from an allowed domain.

Example: Credits must be an integer from 1 to 6.

2. **Key constraint:** Candidate-key values must be unique.
3. **Entity integrity:** A primary-key attribute cannot be **NULL**.
4. **Referential integrity:** A foreign-key value must match an existing referenced key value or be **NULL** if permitted.
5. **Semantic/business constraint:** Organization-specific rule.

Example: A student's CGPA must be between 0 and 10.

Rapid Revision

24. One-Line Definitions

- **Database:** Organized collection of logically related data.
- **DBMS:** Software for defining, storing, manipulating and controlling a database.
- **Metadata:** Data describing the structure and properties of other data.
- **Schema:** Overall database design.
- **Instance:** Data stored at a particular moment.
- **Data model:** Concepts used to describe data, relationships and constraints.
- **Data independence:** Ability to change one schema level without changing higher levels.
- **Entity:** Distinguishable real-world object.
- **Attribute:** Property of an entity or relationship.
- **Relationship:** Association among entities.
- **Candidate key:** Minimal super key.
- **Primary key:** Candidate key selected for identification.
- **Weak entity:** Entity that depends on an owner for identification and existence.
- **Cardinality:** Maximum number of relationship participations.
- **Participation:** Minimum participation requirement in a relationship.
- **Specialization:** Top-down creation of subclasses.
- **Generalization:** Bottom-up creation of a superclass.
- **Aggregation:** Treating a relationship as a higher-level entity.

25. Must-Remember Differences

DDL vs DML

DDL	DML
Defines structure	Retrieves/modifies records
CREATE, ALTER, DROP	SELECT, INSERT, UPDATE, DELETE
Affects schema	Affects database state

Primary Key vs Foreign Key

Primary Key	Foreign Key
Uniquely identifies a row	References a key in another/same table
Unique and not null	May repeat; may be null if allowed
One selected primary key per table	A table may have many foreign keys
Enforces entity integrity	Enforces referential integrity

Cardinality vs Participation

Cardinality	Participation
Maximum participation	Minimum participation
1:1, 1:N, M:N	Total or partial

AKTU Exam Preparation

26. Important Long-Answer Questions

1. Explain the three-schema architecture of DBMS with a diagram. How does it provide data independence?
2. Compare a traditional file-processing system with a DBMS.
3. Explain physical and logical data independence with suitable examples.
4. Describe DBMS structure and explain the query processor and storage manager.
5. Explain different data models and compare hierarchical, network and relational models.
6. What is an ER model? Explain different ER symbols and types of attributes.
7. Explain super key, candidate key, primary key, alternate key, composite key and foreign key with examples.
8. Explain mapping cardinalities and participation constraints with examples.
9. What is a weak entity? Compare strong and weak entity types with an ER diagram.
10. Explain specialization and generalization. Discuss disjointness and completeness constraints.
11. Explain aggregation and category/union type in the EER model.
12. Convert an ER diagram containing 1:1, 1:N and M:N relationships into relational schemas.
13. Design an ER diagram for a university, banking, hospital or library database and map it to relations.
14. Explain the role and responsibilities of a DBA.
15. Explain one-tier, two-tier and three-tier DBMS architectures.

27. Important Short-Answer Questions

1. Define metadata and data dictionary.
2. Differentiate schema and instance.
3. Why is logical data independence harder to achieve?
4. What is a derived attribute? Give an example.
5. What is a recursive relationship?
6. Define total and partial participation.
7. What is a partial key?
8. Define a ternary relationship.
9. What is an associative entity?
10. Differentiate DDL and DML.
11. What is a candidate key?
12. State the entity and referential integrity rules.
13. What is aggregation?
14. What is an overlapping specialization?
15. What is a category in the EER model?

28. Practice Questions

Question 1

A university has students and courses. A student may register for many courses, and each course may have many students. The university stores registration date and grade.

Answer outline:

- Entities: STUDENT and COURSE
- Relationship: REGISTRATION
- Cardinality: M:N
- Relationship attributes: RegistrationDate, Grade
- Mapping: Create REGISTRATION(StudentId, CourseId, RegistrationDate, Grade)

Question 2

Every employee works in exactly one department, while a department can employ many employees.

Answer outline:

- Relationship cardinality: DEPARTMENT 1:N EMPLOYEE
- EMPLOYEE participation: total
- Put `DeptId` as a foreign key in EMPLOYEE

Question 3

An employee can have dependents. A dependent has a name, date of birth and relationship, but no independent identifier.

Answer outline:

- DEPENDENT is weak
- EMPLOYEE is owner
- DependentName is partial key
- Primary key after mapping: (`EmployeeId`, `DependentName`)

29. Suggested Answer-Writing Format

For a 7–10 mark theory question:

1. Begin with a precise definition.
2. Draw a labeled diagram where applicable.
3. Explain each component in separate points.
4. Add a suitable example.
5. Include a comparison table if relevant.
6. End with advantages, significance or a brief conclusion.

For an ER-design question:

1. Identify entity types.
2. List attributes and underline keys.
3. Identify relationships and their degree.
4. Mark cardinality and participation.
5. Identify weak entities and partial keys.
6. Check for multivalued/derived attributes.
7. Convert the ER design into relations.
8. Mark primary and foreign keys clearly.

30. Five-Minute Revision Sheet

DBMS = database + management software

Main goals = low redundancy, consistency, sharing, security, integrity, recovery

Three levels:

External Conceptual Internal

Data independence:

Logical = change conceptual without changing views/programs

Physical = change internal without changing conceptual

Schema = design; Instance = current data

Keys:

Super Candidate Primary/Alternate

Foreign key references another key

ER:

Entity = rectangle

Relationship = diamond

Attribute = ellipse

Weak entity = double rectangle

Multivalued = double ellipse

Derived = dashed ellipse

Total participation = double line

Cardinality = maximum: 1:1, 1:N, M:N

Participation = minimum: total or partial

Weak entity key = owner key + partial key

Specialization = top-down

Generalization = bottom-up

Aggregation = relationship treated as higher-level entity

Mapping:

1:N foreign key on N side

M:N new relation

Multivalued attribute new relation

Weak entity owner key + partial key

Final Exam Tip

Do not only memorize ER definitions. Practice identifying entities, keys, cardinalities and participation constraints from written requirements, because AKTU questions often combine theory with ER design and relational mapping.

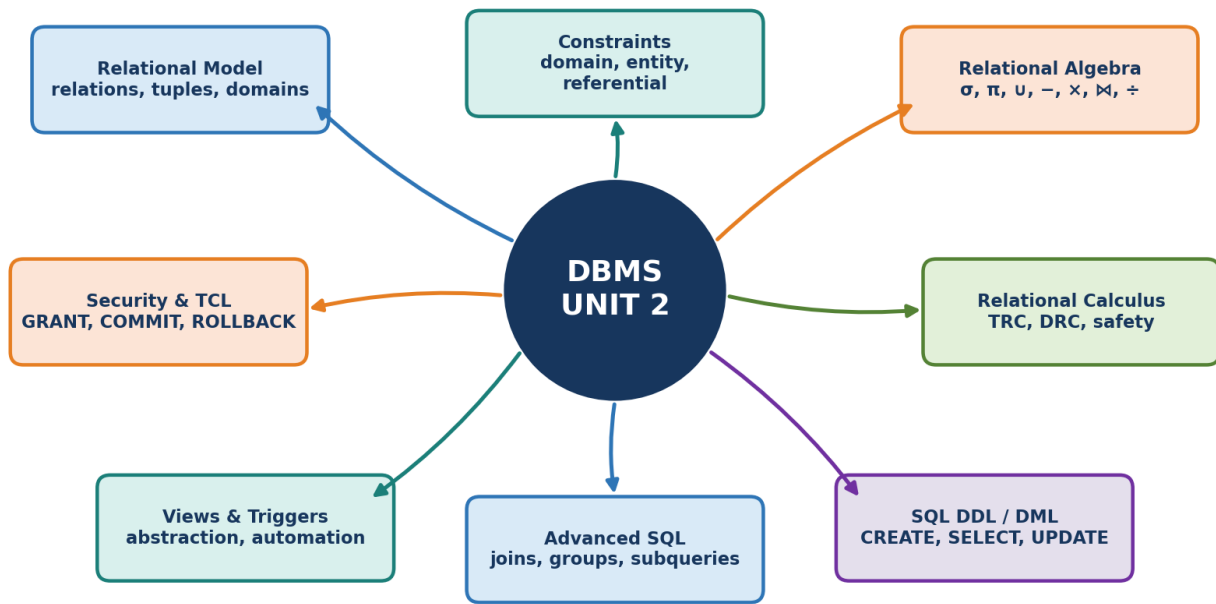
DBMS Unit 2 Notes

B.Tech 3rd Year — AKTU (Exam-Oriented)

Scope: These notes cover the topics commonly taught in AKTU DBMS Unit 2: the relational data model, relational constraints, relational algebra, tuple and domain relational calculus, SQL data definition and manipulation, joins, nested queries, views, constraints, triggers, authorization and transactions.

Unit 2 Visual Mind Map

Unit 2 Mind Map — Relational Model, Algebra and SQL



Unit 2 mind map showing relational model, algebra, calculus, SQL, views, triggers and security

Follow the map from relational concepts and constraints to formal query languages, then connect those ideas to practical SQL.

1. Relational Data Model

The **relational model**, proposed by E. F. Codd, represents a database as a collection of relations or tables. It provides a simple logical structure and a strong mathematical foundation based on set theory and predicate logic.

1.1 Basic Terminology

Consider the relation:

STUDENT(StudentId, Name, Branch, Semester, CGPA)

Term	Meaning	Example
Relation	A table	STUDENT
Tuple	A row in a relation	(101, 'Aman', 'CSE', 5, 8.4)
Attribute	A named column	Branch
Domain	Allowed set of values for an attribute	CGPA: 0.0 to 10.0
Relation schema	Name and attributes of a relation	STUDENT(StudentId, Name, Branch, Semester, CGPA)
Relation instance/state	Current set of tuples	Rows currently stored in STUDENT
Degree/arity	Number of attributes	Degree of STUDENT = 5
Cardinality	Number of tuples	200 students means cardinality = 200
Database schema	Set of all relation schemas	STUDENT, COURSE, ENROLLMENT, etc.

Important: In the relational model, “cardinality” of a relation means the number of rows. In ER modeling, cardinality describes relationship ratios such as 1:N or M:N.

1.2 Properties of a Relation

In the pure relational model:

1. A relation has a unique name.
2. Each attribute has a unique name within the relation.
3. Every cell contains one atomic value.
4. Values in a column come from the same domain.
5. Duplicate tuples are not allowed.
6. The order of rows is immaterial.
7. The order of columns is immaterial.
8. Each tuple represents one fact about an entity or relationship.

SQL tables may allow duplicate rows unless prevented by a key or **DISTINCT**; the theoretical relational model treats a relation as a set and therefore disallows duplicates.

1.3 Domain

A **domain** is the set of permitted atomic values for an attribute. A domain includes the data type, format and permitted range.

Examples:

- **StudentId**: positive integers
- **Branch**: {CSE, IT, ECE, ME, CE}
- **CGPA**: decimal values from 0 to 10
- **Email**: character strings satisfying an email format

1.4 NULL Value

NULL may represent:

- Unknown value
- Missing value
- Not yet available value
- Not applicable value

NULL is not zero, an empty string or a blank space. Comparisons with NULL use **IS NULL** or **IS NOT NULL**, not **= NULL**.

SQL uses **three-valued logic**: TRUE, FALSE and UNKNOWN. For example, **Salary > 50000** is UNKNOWN when Salary is NULL.

2. Keys in the Relational Model

Consider:

STUDENT(StudentId, UniversityRollNo, Email, Name, DeptId)

2.1 Super Key

A **super key** is any set of attributes that uniquely identifies a tuple.

Examples: {StudentId}, {Email}, {StudentId, Name}.

2.2 Candidate Key

A **candidate key** is a minimal super key. Removing any attribute from it destroys uniqueness.

Examples: {StudentId}, {UniversityRollNo}, {Email}.

2.3 Primary Key

The **primary key** is the candidate key selected as the main identifier. It must be unique and cannot contain **NULL**.

2.4 Alternate Key

Candidate keys not chosen as the primary key are **alternate keys**.

2.5 Composite Key

A **composite key** contains more than one attribute.

Example: (StudentId, CourseId, Semester) in ENROLLMENT.

2.6 Foreign Key

A **foreign key** is an attribute or set of attributes whose values reference a candidate key, usually the primary key, of another or the same relation.

```
DEPARTMENT(DeptId, DeptName)
STUDENT(StudentId, Name, DeptId)
```

STUDENT.DeptId references DEPARTMENT.DeptId.

2.7 Surrogate Key

A **surrogate key** is an artificial identifier generated by the system, such as an auto-incremented integer.

3. Relational Integrity Constraints

Integrity constraints maintain correctness and consistency.

3.1 Domain Constraint

Every attribute value must be atomic and belong to its declared domain.

```
CGPA DECIMAL(4,2) CHECK (CGPA BETWEEN 0 AND 10)
```

3.2 Key Constraint

No two distinct tuples may have the same value for a candidate key.

3.3 Entity Integrity Constraint

A primary-key attribute cannot be **NULL** because every tuple must be identifiable.

3.4 Referential Integrity Constraint

A non-null foreign-key value must match an existing candidate-key value in the referenced relation.

If a referenced department does not exist, a student cannot be assigned to it.

3.5 Semantic/Business Constraints

Rules specific to an organization.

Examples:

- A student's CGPA must be between 0 and 10.
- A course must carry at least one credit.
- An employee's joining date cannot be in the future.

3.6 Actions on Referenced Rows

When a referenced row is updated or deleted, SQL can specify:

- **RESTRICT/NO ACTION**: Reject the operation.
- **CASCADE**: Propagate update or deletion to referencing rows.

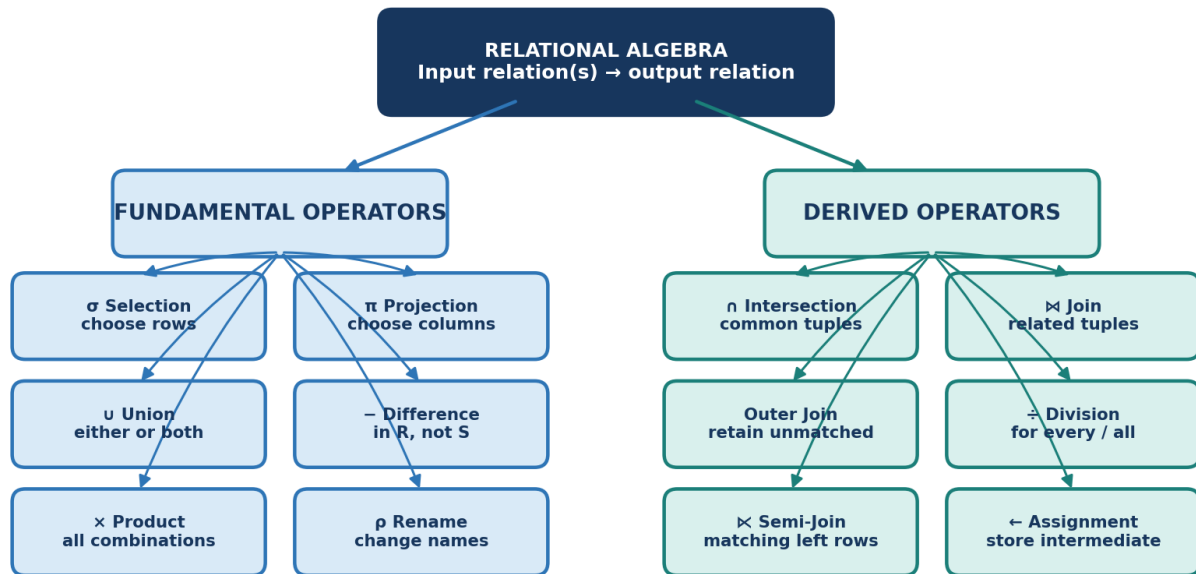
- **SET NULL:** Set the foreign key to **NULL**.
- **SET DEFAULT:** Set the foreign key to its default value.

FOREIGN KEY (DeptId) REFERENCES Department(DeptId)
ON UPDATE CASCADE
ON DELETE SET NULL

Relational Algebra

4. Introduction to Relational Algebra

Relational Algebra — Operator Map



Closure property: every operator produces another relation, so operations can be composed.

Relational algebra operator map separating fundamental and derived operators

Relational algebra is a procedural query language. A query specifies both what result is required and a sequence of operations to obtain it.

4.1 Closure Property

The input and output of every relational algebra operation are relations. Therefore, operations can be nested or composed.

4.2 Sample Database Used in Examples

STUDENT(Sid, Sname, Branch, Age)
COURSE(Cid, Cname, Credits)
ENROLL(Sid, Cid, Grade)

4.3 Classification of Operators

Fundamental operators:

- Selection σ
- Projection π
- Union
- Set difference $-$
- Cartesian product $\times$
- Rename ρ

Derived operators:

- Intersection

- Join
- Division $\div$
- Assignment

5. Fundamental Relational Algebra Operations

5.1 Selection

Selection chooses tuples that satisfy a condition. It is a horizontal subset of a relation.

$\sigma_{\text{condition}}(\text{Relation})$

Example: Find CSE students.

$\sigma_{\text{Branch} = \text{'CSE'}}(\text{STUDENT})$

Properties:

- Degree remains unchanged.
- Cardinality may decrease or remain the same.
- Selections are commutative:

$\sigma_1(\sigma_2(R)) = \sigma_2(\sigma_1(R))$

5.2 Projection

Projection chooses specified attributes. It is a vertical subset and removes duplicate tuples in pure relational algebra.

$\pi_{\text{attribute-list}}(\text{Relation})$

Example: Display names and branches.

$\pi_{\text{Name, Branch}}(\text{STUDENT})$

Properties:

- Degree is equal to the number of selected attributes.
- Duplicate result tuples are eliminated.
- Projections are not generally commutative.

5.3 Union

Union returns tuples present in either relation or both.

$R \cup S$

For union, R and S must be **union-compatible**:

1. They must have the same degree.
2. Corresponding attributes must have compatible domains.

5.4 Set Difference

Set difference returns tuples present in R but not in S.

$R - S$

R and S must be union-compatible. Difference is not commutative:

$R - S \neq S - R$

5.5 Cartesian Product

The Cartesian product combines every tuple of R with every tuple of S.

$R \times S$

If R has m tuples and S has n tuples, $R \times S$ has $m \times n$ tuples. Its degree is $\text{degree}(R) + \text{degree}(S)$.

Example:

$\text{STUDENT} \times \text{COURSE}$

A Cartesian product is commonly followed by selection to form a join.

5.6 Rename

Rename changes a relation name or attribute names.

$\rho_{\text{NewName}}(R)$
 $\rho_{\text{NewName}}(A_1, A_2, \dots)(R)$

It is essential for self-joins and for avoiding ambiguous attribute names.

Example:

$\rho_{S1}(\text{STUDENT})$

6. Derived Relational Algebra Operations

6.1 Intersection

Intersection returns tuples common to both relations.

$R \cap S$

It can be expressed as:

$R \cap S = R - (R - S)$

6.2 Theta Join

A theta join combines related tuples using any comparison condition.

$R \bowtie_{\text{condition } S} S = \sigma_{\text{condition}}(R \times S)$

Example:

$\text{STUDENT} \bowtie_{\text{STUDENT.Sid} = \text{ENROLL.Sid}} \text{ENROLL}$

6.3 Equi-Join

An **equi-join** is a theta join using only equality. Both copies of the join columns normally appear in the result.

6.4 Natural Join

A **natural join** automatically joins relations on all identically named compatible attributes and retains one copy of each common attribute.

$\text{STUDENT} \bowtie_{\text{natural}} \text{ENROLL}$

Natural joins should be used carefully: unrelated columns with the same name may be joined unintentionally.

6.5 Outer Joins

Outer joins retain unmatched tuples by padding missing attributes with **NULL**.

- **Left outer join** : Keeps every tuple from the left relation.
- **Right outer join** : Keeps every tuple from the right relation.
- **Full outer join** : Keeps every tuple from both relations.

6.6 Semi-Join

A semi-join returns only attributes of one relation for tuples having a match in another relation.

$R \ltimes S$

It is useful in distributed query processing because it can reduce transferred data.

6.7 Division

Division answers queries containing “all” or “every.”

If $R(X, Y)$ and $S(Y)$, then $R \div S$ returns X values related to every Y value in S .

Example: Find students enrolled in **all** compulsory courses.

$\pi_{\text{Sid}, \text{Cid}}(\text{ENROLL}) \div \pi_{\text{Cid}}(\text{COMPULSORY_COURSE})$

6.8 Assignment

Assignment stores an intermediate result.

`CSE_STUDENT` $\sigma_{\text{Branch} = \text{'CSE'}}(\text{STUDENT})$

7. Relational Algebra Query Examples

Using:

`STUDENT`(Sid, Sname, Branch, Age)
`COURSE`(Cid, Cname, Credits)
`ENROLL`(Sid, Cid, Grade)

Query 1: Names of all CSE students

$\pi_{\text{Sname}}(\sigma_{\text{Branch} = \text{'CSE'}}(\text{STUDENT}))$

Query 2: Names of students enrolled in course C101

$\pi_{\text{Sname}}(\text{STUDENT} \bowtie_{\text{STUDENT.Sid} = \text{ENROLL.Sid}} (\sigma_{\text{Cid} = \text{'C101'}}(\text{ENROLL})))$

Query 3: Course names taken by student 101

$\pi_{\text{Cname}}((\sigma_{\text{Sid} = 101}(\text{ENROLL})) \bowtie \text{COURSE})$

Query 4: Students who are not enrolled in any course

$\pi_{\text{Sid}}(\text{STUDENT}) - \pi_{\text{Sid}}(\text{ENROLL})$

To obtain names:

$\pi_{\text{Sname}}(\text{STUDENT} \bowtie_{\pi_{\text{Sid}}(\text{STUDENT}) - \pi_{\text{Sid}}(\text{ENROLL})})$

Query 5: Students enrolled in both C101 and C102

$\pi_{\text{Sid}}(\sigma_{\text{Cid} = \text{'C101'}}(\text{ENROLL})) \cap \pi_{\text{Sid}}(\sigma_{\text{Cid} = \text{'C102'}}(\text{ENROLL}))$

Query 6: Students enrolled in every course

$\pi_{\text{Sid}, \text{Cid}}(\text{ENROLL}) \div \pi_{\text{Cid}}(\text{COURSE})$

Query 7: Pairs of students from the same branch

$\pi_{\text{S1.Sname}, \text{S2.Sname}}(\sigma_{\text{S1.Branch} = \text{S2.Branch} \wedge \text{S1.Sid} < \text{S2.Sid}}(\rho_{\text{S1}}(\text{STUDENT}) \times \rho_{\text{S2}}(\text{STUDENT})))$

Relational Calculus

8. Relational Algebra vs Relational Calculus

Relational calculus is a **non-procedural/declarative** formal query language based on predicate logic. It specifies what result is wanted, not an execution procedure.

Two forms are:

1. Tuple Relational Calculus (TRC)
2. Domain Relational Calculus (DRC)

Relational Algebra	Relational Calculus
Procedural	Non-procedural
Specifies what and how	Specifies what only
Based on algebraic operators	Based on predicate logic
Uses relations as operands	Uses tuple/domain variables
Foundation for query execution	Foundation for declarative query languages

For safe expressions, relational algebra, TRC and DRC have equivalent expressive power. This result is known as **Codd's theorem** or relational completeness equivalence.

9. Tuple Relational Calculus (TRC)

TRC uses tuple variables.

9.1 General Form

$\{ t \mid P(t) \}$

Meaning: the set of all tuples t for which predicate $P(t)$ is true.

9.2 Atomic Formulas

- $t \in R$: tuple t belongs to relation R .
- $t[A] \theta u[B]$: comparison between tuple attributes.
- $t[A] \theta \text{constant}$: comparison with a constant.

Here, θ may be $=, <, >$ or $\neq$.

9.3 Logical Connectives and Quantifiers

- $\wedge$: AND
- $\vee$: OR
- $\neg$: NOT
- $\exists$: There exists
- $\forall$: For all

9.4 Examples

Find all CSE student tuples:

$\{ t \mid t \in \text{STUDENT} \wedge t.\text{Branch} = \text{'CSE'} \}$

Find names of students enrolled in C101:

$\{ t.\text{Sname} \mid$
 $t \in \text{STUDENT}$
 $\wedge \exists e (e \in \text{ENROLL} \wedge e.\text{Sid} = t.\text{Sid} \wedge e.\text{Cid} = \text{'C101'})$

```
}

```

Find students not enrolled in any course:

```
{ t.Sname |
  t STUDENT
  ¬e (e ENROLL e.Sid = t.Sid)
}
```

9.5 Free and Bound Variables

- A variable is **bound** if it is quantified by $\forall$ or $\exists$.
- A variable is **free** if it is not quantified.
- Free variables determine the attributes in the result.

10. Domain Relational Calculus (DRC)

DRC uses variables that range over individual domain values rather than complete tuples.

10.1 General Form

```
{ <x1, x2, ..., xn> | P(x1, x2, ..., xn) }
```

10.2 Examples

Find names of CSE students:

```
{ <name> |
  sid, age (STUDENT(sid, name, 'CSE', age))
}
```

Find names of students enrolled in C101:

```
{ <name> |
  sid, branch, age, grade (
    STUDENT(sid, name, branch, age)
    ENROLL(sid, 'C101', grade)
  )
}
```

10.3 Safe Expressions

An expression is **safe** if it produces a finite result containing only values from the active database domain. Unsafe expressions can describe infinite sets.

Unsafe example:

```
{ <x> | ¬STUDENT_ID(x) }
```

This may include infinitely many values not appearing in the database.

TRC vs DRC

TRC	DRC		
Variables represent tuples	Variables represent domain values		
Form: $\{t\}$	$P(t)$	Form: $\{<x_1, \dots, x_n>\}$	$P(\dots)$
Attribute access uses t.A	Relation predicate lists domain variables		
Closer to row-based thinking	Closer to value-based logical notation		

Structured Query Language (SQL)

11. SQL Overview

SQL is a declarative language used to define, query, modify and control relational databases.

11.1 SQL Command Categories

Category	Purpose	Commands
DDL	Defines schema objects	CREATE, ALTER, DROP, TRUNCATE
DML	Changes data	INSERT, UPDATE, DELETE
DQL	Retrieves data	SELECT
DCL	Controls privileges	GRANT, REVOKE
TCL	Controls transactions	COMMIT, ROLLBACK, SAVEPOINT

11.2 Common SQL Data Types

- **CHAR(n)**: Fixed-length character string
- **VARCHAR(n)**: Variable-length character string
- **INTEGER/INT**: Integer
- **DECIMAL(p,s)**: Exact numeric value
- **REAL, FLOAT**: Approximate numeric value
- **DATE**: Calendar date
- **TIME**: Time of day
- **TIMESTAMP**: Date and time
- **BOOLEAN**: True/false where supported

12. SQL Data Definition Language

12.1 CREATE TABLE

```
CREATE TABLE Department (
  DeptId   INT PRIMARY KEY,
  DeptName VARCHAR(50) NOT NULL UNIQUE
);
```

```
CREATE TABLE Student (
  StudentId INT PRIMARY KEY,
  Name      VARCHAR(60) NOT NULL,
  Email     VARCHAR(100) UNIQUE,
  Branch    VARCHAR(10) NOT NULL,
  Semester  INT DEFAULT 1,
  CGPA      DECIMAL(4,2),
  DeptId    INT,
  CONSTRAINT chk_semester CHECK (Semester BETWEEN 1 AND 8),
  CONSTRAINT chk_cgpa CHECK (CGPA BETWEEN 0 AND 10),
  CONSTRAINT fk_student_department
    FOREIGN KEY (DeptId) REFERENCES Department(DeptId)
);
```

12.2 CREATE TABLE with Composite Primary Key

```
CREATE TABLE Enrollment (
  StudentId INT,
  CourseId VARCHAR(10),
  Semester INT,
  Grade VARCHAR(2),
  PRIMARY KEY (StudentId, CourseId, Semester),
  FOREIGN KEY (StudentId) REFERENCES Student(StudentId),
  FOREIGN KEY (CourseId) REFERENCES Course(CourseId)
);
```

12.3 ALTER TABLE

```
ALTER TABLE Student ADD Phone VARCHAR(15);
ALTER TABLE Student ALTER COLUMN Name TYPE VARCHAR(100);
ALTER TABLE Student DROP COLUMN Phone;
ALTER TABLE Student ADD CONSTRAINT uq_email UNIQUE (Email);
```

Exact syntax may vary across MySQL, PostgreSQL, Oracle and SQL Server.

12.4 DROP and TRUNCATE

```
DROP TABLE Student;
TRUNCATE TABLE Student;
```

DELETE	TRUNCATE	DROP
DML command	Usually treated as DDL	DDL command
Removes selected/all rows	Removes all rows	Removes entire table object
WHERE is allowed	No WHERE	No WHERE
Row-by-row logging is common	Usually minimally logged	Definition and data removed
Structure remains	Structure remains	Structure is removed
Triggers may execute	Behavior varies by DBMS	Table no longer exists

Transaction and rollback behavior of TRUNCATE varies by DBMS.

13. SQL Data Manipulation

13.1 INSERT

```
INSERT INTO Department (DeptId, DeptName)
VALUES (10, 'Computer Science');
```

```
INSERT INTO Student (StudentId, Name, Branch, Semester, CGPA, DeptId)
VALUES (101, 'Aman', 'CSE', 5, 8.40, 10);
```

Insert rows produced by a query:

```
INSERT INTO CSE_Student (StudentId, Name)
SELECT StudentId, Name
FROM Student
WHERE Branch = 'CSE';
```

13.2 UPDATE

```
UPDATE Student
SET CGPA = 8.70
WHERE StudentId = 101;
```

Without a WHERE condition, every row is updated.

13.3 DELETE

```
DELETE FROM Student
WHERE StudentId = 101;
```

Without a WHERE condition, every row is deleted, but the table remains.

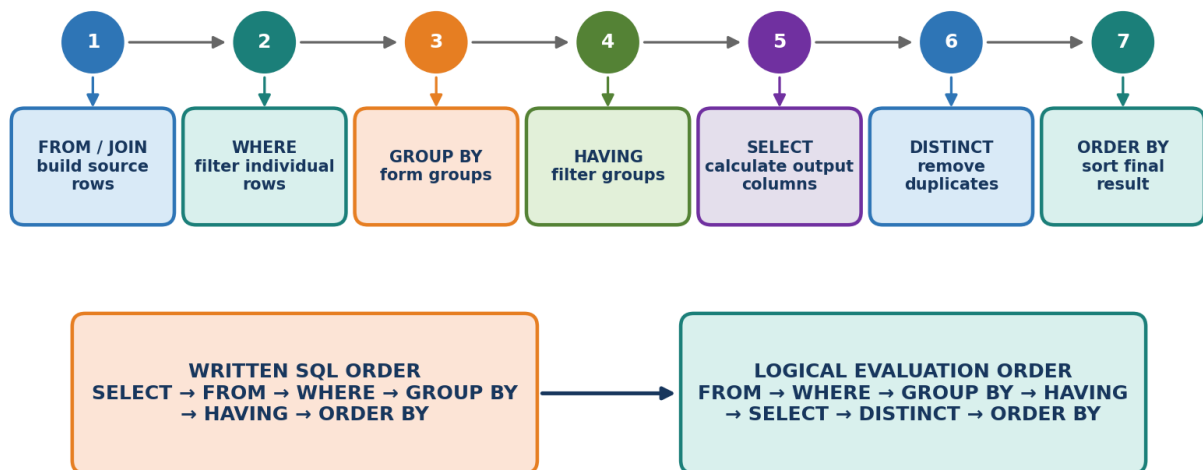
14. Basic SELECT Queries

14.1 General Form

```
SELECT [DISTINCT] column_expression, ...
FROM table_expression
WHERE row_condition
GROUP BY grouping_columns
HAVING group_condition
ORDER BY sort_columns;
```

14.2 Logical Query Processing Order

SQL Logical Query Processing Order



Exam key: WHERE filters rows before grouping; HAVING filters groups after grouping.

SQL logical processing flow from FROM and WHERE through GROUP BY, HAVING, SELECT and ORDER BY

A useful conceptual order is:

FROM WHERE GROUP BY HAVING SELECT DISTINCT ORDER BY

14.3 Examples

```
SELECT * FROM Student;
```

```
SELECT Name, CGPA
FROM Student
WHERE Branch = 'CSE';
```

```
SELECT DISTINCT Branch
FROM Student;
```

```
SELECT Name, CGPA
FROM Student
ORDER BY CGPA DESC, Name ASC;
```

14.4 Aliases

```
SELECT s.Name AS StudentName, s.CGPA AS Score
FROM Student AS s;
```

Aliases improve readability and are essential in self-joins.

14.5 Arithmetic Expressions

```
SELECT EmpId, Name, Salary, Salary * 12 AS AnnualSalary
FROM Employee;
```

15. SQL Operators and Predicates

15.1 Comparison Operators

=, <> or !=, <, <=, >, >=

15.2 Logical Operators

AND, OR, NOT

Precedence is generally **NOT**, then **AND**, then **OR**. Use parentheses to make intent clear.

15.3 BETWEEN

```
SELECT * FROM Student
WHERE CGPA BETWEEN 8.0 AND 10.0;
```

BETWEEN includes both boundary values.

15.4 IN

```
SELECT * FROM Student
WHERE Branch IN ('CSE', 'IT', 'ECE');
```

15.5 LIKE

```
SELECT * FROM Student WHERE Name LIKE 'A%';
SELECT * FROM Student WHERE Name LIKE '_a%';
```

- **%**: Any sequence of zero or more characters
- **_**: Exactly one character

15.6 NULL Predicates

```
SELECT * FROM Student WHERE Email IS NULL;
SELECT * FROM Student WHERE Email IS NOT NULL;
```

Never use `Email = NULL`.

16. SQL Functions

16.1 Aggregate Functions

- **COUNT**: Counts rows/non-null values
- **SUM**: Sum of values
- **AVG**: Average of values
- **MIN**: Minimum value
- **MAX**: Maximum value

```
SELECT COUNT(*) AS TotalStudents,
       AVG(CGPA) AS AverageCGPA,
       MAX(CGPA) AS HighestCGPA
FROM Student;
```

Most aggregate functions ignore **NULL**; **COUNT(*)** counts rows.

16.2 Scalar Functions

DBMSs provide functions for:

- **Strings**: **UPPER**, **LOWER**, **LENGTH**, **SUBSTRING**
- **Numbers**: **ROUND**, **ABS**, **CEIL**, **FLOOR**

- Date/time: `CURRENT_DATE`, extraction and date arithmetic
- Null handling: `COALESCE`

```
SELECT UPPER(Name), ROUND(CGPA, 1), COALESCE(Email, 'Not given')
FROM Student;
```

Function names and syntax can vary by DBMS.

17. GROUP BY and HAVING

`GROUP BY` forms groups of rows having the same values. Aggregate functions can then be applied to each group.

```
SELECT Branch, COUNT(*) AS StudentCount, AVG(CGPA) AS AvgCGPA
FROM Student
GROUP BY Branch;
```

`WHERE` filters individual rows before grouping. `HAVING` filters groups after grouping.

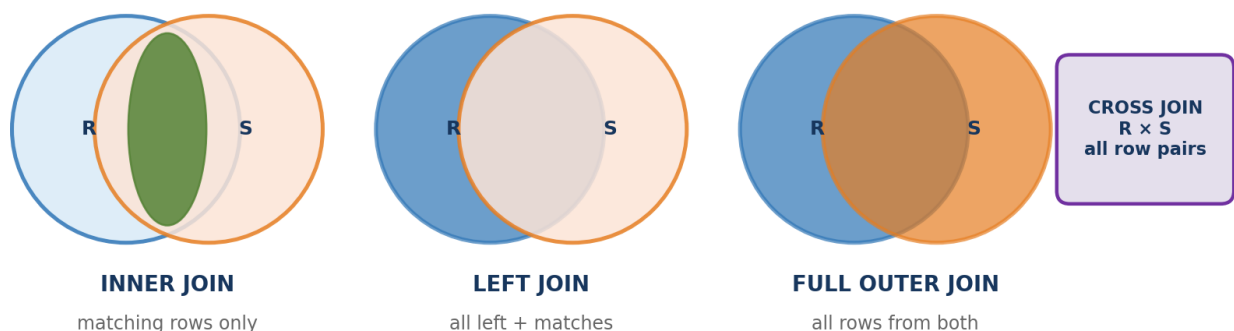
```
SELECT Branch, AVG(CGPA) AS AvgCGPA
FROM Student
WHERE Semester >= 3
GROUP BY Branch
HAVING AVG(CGPA) >= 8.0;
```

WHERE vs HAVING

WHERE	HAVING
Filters rows	Filters groups
Evaluated before grouping	Evaluated after grouping
Aggregate conditions normally not used	Aggregate conditions are commonly used
Works without GROUP BY	Often used with GROUP BY

18. SQL Joins

SQL Join Visualization



RIGHT JOIN is the mirror image of **LEFT JOIN**. A self-join uses two aliases of the same table.

Visual comparison of inner, left, full outer and cross joins

Assume:

```
STUDENT(StudentId, Name, DeptId)
```

```
DEPARTMENT(DeptId, DeptName)
ENROLLMENT(StudentId, CourseId, Grade)
COURSE(CourseId, CourseName)
```

18.1 Inner Join

Returns only matching rows.

```
SELECT s.Name, d.DeptName
FROM Student AS s
JOIN Department AS d
ON s.DeptId = d.DeptId;
```

18.2 Equi-Join

An inner join whose condition uses equality.

```
SELECT s.Name, e.CourseId
FROM Student s, Enrollment e
WHERE s.StudentId = e.StudentId;
```

Explicit JOIN ... ON syntax is usually clearer than comma joins.

18.3 Natural Join

```
SELECT StudentId, Name, DeptName
FROM Student NATURAL JOIN Department;
```

It automatically joins all same-named columns. Avoid it when schema changes could accidentally alter the join condition.

18.4 Left Outer Join

Returns all left-side rows and matching right-side rows. Missing right-side values become **NULL**.

```
SELECT s.StudentId, s.Name, e.CourseId
FROM Student s
LEFT JOIN Enrollment e
ON s.StudentId = e.StudentId;
```

This can find students with no enrollment:

```
SELECT s.StudentId, s.Name
FROM Student s
LEFT JOIN Enrollment e
ON s.StudentId = e.StudentId
WHERE e.StudentId IS NULL;
```

18.5 Right and Full Outer Join

```
SELECT ... FROM R RIGHT JOIN S ON condition;
SELECT ... FROM R FULL OUTER JOIN S ON condition;
```

A full outer join keeps all matched and unmatched rows from both sides. Support varies among DBMSs.

18.6 Cross Join

```
SELECT * FROM Student CROSS JOIN Course;
```

It returns the Cartesian product.

18.7 Self-Join

A table is joined with itself using aliases.

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employee e
LEFT JOIN Employee m
ON e.ManagerId = m.EmpId;
```

19. Set Operations in SQL

19.1 UNION

Combines results and removes duplicates.

```
SELECT Email FROM Student
UNION
SELECT Email FROM Teacher;
```

19.2 UNION ALL

Combines results and retains duplicates. It is usually faster because duplicate elimination is unnecessary.

19.3 INTERSECT

Returns rows common to both query results.

19.4 EXCEPT/MINUS

Returns rows in the first result but not the second. PostgreSQL and SQL Server use **EXCEPT**; Oracle traditionally uses **MINUS**.

Set-operation queries must return the same number of columns with compatible data types.

20. Nested Queries and Subqueries

A subquery is a query nested inside another SQL statement.

20.1 Single-Row Subquery

```
SELECT Name, CGPA
FROM Student
WHERE CGPA > (SELECT AVG(CGPA) FROM Student);
```

20.2 Multi-Row Subquery with IN

```
SELECT Name
FROM Student
WHERE StudentId IN (
    SELECT StudentId
    FROM Enrollment
    WHERE CourseId = 'C101'
);
```

20.3 EXISTS

EXISTS is true if the subquery returns at least one row.

```
SELECT s.Name
FROM Student s
WHERE EXISTS (
    SELECT 1
    FROM Enrollment e
    WHERE e.StudentId = s.StudentId
);
```

20.4 NOT EXISTS

```
SELECT s.Name
FROM Student s
WHERE NOT EXISTS (
    SELECT 1
    FROM Enrollment e
    WHERE e.StudentId = s.StudentId
);
```

20.5 ANY/SOME and ALL

Salary > ANY (subquery)

Means greater than at least one returned value.

Salary > ALL (subquery)

Means greater than every returned value.

20.6 Correlated Subquery

A correlated subquery refers to a row of the outer query and is conceptually evaluated once per outer row.

```

SELECT s.Name, s.Branch, s.CGPA
FROM Student s
WHERE s.CGPA > (
    SELECT AVG(s2.CGPA)
    FROM Student s2
    WHERE s2.Branch = s.Branch
);

```

This finds students scoring above their own branch average.

Correlated vs Non-Correlated Subquery

Non-Correlated	Correlated
Independent of outer query	Refers to outer-query columns
Can execute independently	Cannot execute meaningfully alone
Conceptually evaluated once	Conceptually evaluated per outer row
Often easier to optimize	May be more expensive

21. Views

A **view** is a named query presented as a virtual table. Its definition is stored; ordinary views generally do not store a separate copy of the result.

21.1 Creating and Using a View

```

CREATE VIEW CSE_Student AS
SELECT StudentId, Name, Semester, CGPA
FROM Student
WHERE Branch = 'CSE';

SELECT * FROM CSE_Student WHERE CGPA >= 8.0;

```

21.2 Advantages of Views

- Restrict access to sensitive columns or rows
- Simplify complex queries
- Provide logical data independence
- Give different users customized representations
- Maintain consistent query logic

21.3 Updating Views

A view is more likely to be updatable when it:

- Is based on one base table
- Includes the key of the base table
- Does not contain grouping or aggregate functions
- Does not use **DISTINCT**, set operations or complex joins
- Does not contain derived columns that prevent mapping

Rules vary by DBMS.

21.4 WITH CHECK OPTION

Prevents an update through a view from creating rows that are no longer visible through the view.

```

CREATE VIEW CSE_Student AS
SELECT StudentId, Name, Branch
FROM Student
WHERE Branch = 'CSE'
WITH CHECK OPTION;

```

21.5 Materialized View

A **materialized view** physically stores a query result and must be refreshed. It improves read performance but consumes storage and may become stale.

Ordinary View	Materialized View
Stores query definition	Stores query result
Always reflects current base data	Requires refresh
Little additional storage	Uses additional storage
Query may be recomputed	Faster for expensive repeated queries

22. SQL Constraints

22.1 NOT NULL

Name VARCHAR(60) NOT NULL

22.2 UNIQUE

Email VARCHAR(100) UNIQUE

22.3 PRIMARY KEY

PRIMARY KEY (StudentId)

It combines uniqueness and non-nullability.

22.4 FOREIGN KEY

FOREIGN KEY (DeptId) REFERENCES Department(DeptId)

22.5 CHECK

CHECK (CGPA BETWEEN 0 AND 10)

22.6 DEFAULT

Semester INT DEFAULT 1

Column-Level vs Table-Level Constraints

- Column-level constraints follow one column definition.
- Table-level constraints appear after column definitions and can involve multiple columns.

```
CONSTRAINT pk_enrollment
PRIMARY KEY (StudentId, CourseId, Semester)
```

Naming constraints simplifies later modification and error diagnosis.

23. Assertions and Triggers

23.1 Assertion

An **assertion** is a database-wide constraint that must always remain true.

Standard SQL concept:

```
CREATE ASSERTION valid_load
CHECK (
  NOT EXISTS (
    SELECT StudentId
    FROM Enrollment
    GROUP BY StudentId, Semester
    HAVING COUNT(*) > 8
  )
);
```

Many commercial DBMSs do not implement **CREATE ASSERTION**; triggers or application logic are used instead.

23.2 Trigger

A **trigger** is a stored action automatically executed when a specified event occurs.

Trigger components:

1. **Event:** INSERT, UPDATE or DELETE
2. **Timing:** BEFORE, AFTER or INSTEAD OF
3. **Condition:** Optional condition deciding whether to act
4. **Action:** SQL statements executed by the trigger
5. **Granularity:** Row-level or statement-level

Generic example:

```
CREATE TRIGGER check_cgpa
BEFORE INSERT OR UPDATE ON Student
FOR EACH ROW
BEGIN
    IF NEW.CGPA < 0 OR NEW.CGPA > 10 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'CGPA must be between 0 and 10';
    END IF;
END;
```

Trigger syntax differs significantly among DBMSs.

Uses of Triggers

- Enforce complex business rules
- Maintain audit logs
- Automatically derive values
- Maintain summary data
- Prevent invalid changes

Disadvantages of Triggers

- Hidden execution can make behavior difficult to understand
- Cascading triggers are hard to debug
- Poorly written triggers reduce performance
- DBMS-specific syntax reduces portability

24. SQL Authorization and Security

24.1 Privileges

Common object privileges include:

- SELECT
- INSERT
- UPDATE
- DELETE
- REFERENCES
- EXECUTE

24.2 GRANT

```
GRANT SELECT, INSERT ON Student TO faculty_user;
```

```
GRANT UPDATE (CGPA) ON Student TO result_admin;
```

24.3 WITH GRANT OPTION

Allows the recipient to grant the privilege to others.

```
GRANT SELECT ON Student TO faculty_user WITH GRANT OPTION;
```

24.4 REVOKE

```
REVOKE INSERT ON Student FROM faculty_user;
```

Revocation may cascade to privileges granted by that user, depending on DBMS rules.

24.5 Roles

A role groups privileges and can be assigned to users.

```
CREATE ROLE faculty_role;  
GRANT SELECT ON Student TO faculty_role;  
GRANT faculty_role TO faculty_user;
```

Roles simplify administration and support the principle of least privilege.

25. Transaction Control in SQL

A **transaction** is a logical unit of work consisting of one or more database operations.

Example: Transferring money requires debit and credit to succeed together.

25.1 COMMIT

Makes transaction changes permanent.

```
COMMIT;
```

25.2 ROLLBACK

Cancels uncommitted changes.

```
ROLLBACK;
```

25.3 SAVEPOINT

Marks a point for partial rollback.

```
SAVEPOINT before_update;  
UPDATE Student SET CGPA = 9.0 WHERE StudentId = 101;  
ROLLBACK TO before_update;
```

25.4 Basic ACID Reminder

- **Atomicity:** All operations occur or none occur.
- **Consistency:** A transaction preserves valid database rules.
- **Isolation:** Concurrent transactions behave as if executed independently.
- **Durability:** Committed changes survive failures.

Detailed transaction management is often covered in a later unit.

Query Equivalence and Optimization Basics

26. Relational Algebra Equivalences

Equivalent expressions produce the same result but may have different costs.

26.1 Cascade of Selections

$$\sigma_{c1 \text{ AND } c2}(R) = \sigma_{c1}(\sigma_{c2}(R))$$

26.2 Commutativity of Selection

$$\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R))$$

26.3 Cascade of Projections

If $L2$ is a subset of $L1$:

$$\pi_{L2}(\pi_{L1}(R)) = \pi_{L2}(R)$$

26.4 Join Commutativity

$$R \bowtie S = S \bowtie R$$

26.5 Join Associativity

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

26.6 Selection Pushdown

Apply selective conditions as early as possible to reduce intermediate tuples.

Instead of:

$$\sigma_{R.A = 10}(R \bowtie S)$$

Use:

$$(\sigma_{R.A = 10}(R)) \bowtie S$$

when the condition uses only attributes of R .

26.7 Projection Pushdown

Keep only required attributes early, while retaining join attributes. This reduces intermediate relation width.

Worked SQL Example

27. University Schema and Queries

27.1 Schema

```
CREATE TABLE Student (  
    StudentId INT PRIMARY KEY,  
    Name VARCHAR(60) NOT NULL,  
    Branch VARCHAR(10) NOT NULL,  
    Semester INT CHECK (Semester BETWEEN 1 AND 8),  
    CGPA DECIMAL(4,2) CHECK (CGPA BETWEEN 0 AND 10)  
);  
  
CREATE TABLE Course (  
    CourseId VARCHAR(10) PRIMARY KEY,  
    CourseName VARCHAR(80) NOT NULL,  
    Credits INT CHECK (Credits BETWEEN 1 AND 6)  
);  
  
CREATE TABLE Enrollment (  
    StudentId INT,  
    CourseId VARCHAR(10),  
    Semester INT,  
    Grade VARCHAR(2),  
    PRIMARY KEY (StudentId, CourseId, Semester),  
    FOREIGN KEY (StudentId) REFERENCES Student(StudentId),  
    FOREIGN KEY (CourseId) REFERENCES Course(CourseId)  
);
```

27.2 Find Students Taking DBMS

```
SELECT DISTINCT s.StudentId, s.Name  
FROM Student s  
JOIN Enrollment e ON e.StudentId = s.StudentId  
JOIN Course c ON c.CourseId = e.CourseId  
WHERE c.CourseName = 'Database Management Systems';
```

27.3 Count Students in Each Course

```
SELECT c.CourseId, c.CourseName, COUNT(e.StudentId) AS StudentCount  
FROM Course c  
LEFT JOIN Enrollment e ON e.CourseId = c.CourseId  
GROUP BY c.CourseId, c.CourseName;
```

The left join includes courses with zero enrollments.

27.4 Find the Highest-CGPA Student in Each Branch

```
SELECT s.Name, s.Branch, s.CGPA  
FROM Student s  
WHERE s.CGPA = (  
    SELECT MAX(s2.CGPA)  
    FROM Student s2  
    WHERE s2.Branch = s.Branch  
);
```

27.5 Find Students Enrolled in Every Course

```
SELECT s.StudentId, s.Name  
FROM Student s  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM Course c  
    WHERE NOT EXISTS (  
        SELECT 1  
        FROM Enrollment e  
        WHERE e.StudentId = s.StudentId  
        AND e.CourseId = c.CourseId  
    )  
);
```

The nested **NOT EXISTS** pattern expresses relational division.

27.6 Find Students with No Enrollment

```
SELECT s.StudentId, s.Name
FROM Student s
WHERE NOT EXISTS (
  SELECT 1
  FROM Enrollment e
  WHERE e.StudentId = s.StudentId
);
```

Rapid Revision

28. One-Line Definitions

- **Relation:** A set of tuples sharing the same attributes.
- **Tuple:** One row of a relation.
- **Attribute:** Named column of a relation.
- **Domain:** Permitted atomic values of an attribute.
- **Degree:** Number of attributes in a relation.
- **Cardinality:** Number of tuples in a relation.
- **Candidate key:** Minimal set of attributes uniquely identifying a tuple.
- **Foreign key:** Attribute set referencing a candidate key.
- **Entity integrity:** A primary key cannot be null.
- **Referential integrity:** A non-null foreign key must match a referenced key.
- **Relational algebra:** Procedural formal query language.
- **Relational calculus:** Declarative formal query language.
- **Selection:** Chooses rows.
- **Projection:** Chooses columns and removes duplicates in relational algebra.
- **Join:** Combines related tuples.
- **Division:** Answers "for all" queries.
- **View:** Named query presented as a virtual table.
- **Trigger:** Stored action automatically fired by a database event.
- **Transaction:** Logical unit of database work.

29. Must-Remember Comparisons

Selection vs Projection

Selection	Projection
Chooses rows	Chooses columns
Horizontal subset	Vertical subset
Uses a condition	Uses an attribute list
Degree unchanged	Degree may decrease
Symbol σ	Symbol π

DELETE vs DROP

DELETE	DROP
Removes rows	Removes database object
DML	DDL
WHERE allowed	WHERE not allowed
Table structure remains	Table structure removed

JOIN vs Subquery

Join	Subquery
Combines rows from relations	Embeds one query within another
Often clear for retrieving columns from several tables	Natural for membership, existence and aggregate comparisons
Optimizer may transform it	Optimizer may transform it into a join

Primary Key vs UNIQUE

Primary Key	UNIQUE
Main row identifier	Additional uniqueness rule
One per table	Multiple allowed per table
Cannot contain NULL	NULL treatment varies by DBMS
Enforces entity integrity	Enforces uniqueness only

AKTU Exam Preparation

30. Important Long-Answer Questions

1. Explain the relational model and its basic terminology with an example.
2. Explain domain, key, entity-integrity and referential-integrity constraints.
3. Describe fundamental and derived relational algebra operations with examples.
4. Explain different joins in relational algebra and SQL.
5. What is relational division? Explain with a suitable query.
6. Compare relational algebra, tuple relational calculus and domain relational calculus.
7. Explain free variables, bound variables and safe expressions in relational calculus.
8. Write relational algebra expressions for a given university or employee database.
9. Explain SQL DDL and DML commands with examples.
10. Explain aggregate functions, **GROUP BY** and **HAVING** with examples.
11. Explain nested and correlated SQL queries with suitable examples.
12. What is a view? Discuss advantages, limitations and view updatability.
13. Explain SQL constraints and referential actions.
14. What is a trigger? Explain trigger components, applications and limitations.
15. Explain SQL authorization using privileges, **GRANT**, **REVOKE** and roles.
16. Explain set operations in relational algebra and SQL.
17. Discuss query equivalence and selection/projection pushdown.
18. Write SQL queries for a given relational schema involving joins, grouping and subqueries.

31. Important Short-Answer Questions

1. Define degree and cardinality of a relation.
2. What is union compatibility?
3. State the closure property of relational algebra.
4. Differentiate selection and projection.
5. Define natural join and equi-join.
6. What is a semi-join?
7. Which relational operation answers “all” queries?
8. Define a safe relational-calculus expression.
9. Differentiate TRC and DRC.
10. What is three-valued logic in SQL?
11. Differentiate **WHERE** and **HAVING**.
12. Differentiate **UNION** and **UNION ALL**.
13. What is a correlated subquery?
14. What is **WITH CHECK OPTION**?
15. Differentiate a view and a materialized view.
16. What is a trigger?
17. What is **WITH GRANT OPTION**?
18. State the entity-integrity rule.

19. What does **ON DELETE CASCADE** do?
20. Why should **NATURAL JOIN** be used carefully?

32. Practice Problems

Problem 1: Relational Algebra

Given:

```
EMPLOYEE(Eid, Ename, DeptId, Salary)
DEPARTMENT(DeptId, DeptName)
PROJECT(Pid, Pname, DeptId)
WORKS_ON(Eid, Pid, Hours)
```

Write expressions for:

1. Names of employees in the CSE department.
2. Names of employees working on project P10.
3. Employees who do not work on any project.
4. Employees who work on all projects.

Answer outline:

1. $\pi_{\text{Ename}}(\text{EMPLOYEE} \bowtie_{\text{DeptName}='CSE'}(\text{DEPARTMENT}))$
2. $\pi_{\text{Ename}}(\text{EMPLOYEE} \bowtie_{\text{Pid}='P10'}(\text{WORKS_ON}))$
3. $\pi_{\text{Eid}, \text{Ename}}(\text{EMPLOYEE} - (\pi_{\text{Eid}}(\text{EMPLOYEE}) \cap \pi_{\text{Eid}}(\text{WORKS_ON})))$
4. $\pi_{\text{Eid}, \text{Pid}}(\text{WORKS_ON}) \div \pi_{\text{Pid}}(\text{PROJECT})$

Problem 2: SQL

Find departments whose average salary is greater than 60000 and that have at least five employees.

```
SELECT d.DeptId, d.DeptName, AVG(e.Salary) AS AvgSalary
FROM Department d
JOIN Employee e ON e.DeptId = d.DeptId
GROUP BY d.DeptId, d.DeptName
HAVING AVG(e.Salary) > 60000
AND COUNT(*) >= 5;
```

Problem 3: SQL

Find the second-highest distinct salary.

```
SELECT MAX(Salary) AS SecondHighestSalary
FROM Employee
WHERE Salary < (SELECT MAX(Salary) FROM Employee);
```

Problem 4: Universal Query

Find employees who work on every project controlled by their department.

Hint: Use double **NOT EXISTS** and correlate both department and employee.

33. Common Exam Mistakes

1. Confusing relation cardinality with ER cardinality.
2. Forgetting union compatibility for union, intersection and difference.
3. Writing projection before retaining attributes needed by a join.
4. Forgetting that pure relational algebra removes duplicate tuples.
5. Using **= NULL** instead of **IS NULL**.
6. Using aggregate conditions in **WHERE** instead of **HAVING**.
7. Omitting a join condition and accidentally producing a Cartesian product.
8. Selecting a non-grouped, non-aggregated column in a grouped query.

9. Using **NOT IN** without considering **NULL**; **NOT EXISTS** is often safer.
10. Forgetting the composite primary key of an M:N relationship table.
11. Treating a view as if it always stores data physically.
12. Assuming trigger syntax is identical across DBMSs.

34. Five-Minute Revision Sheet

RELATIONAL MODEL

Relation = table; tuple = row; attribute = column

Degree = number of attributes

Cardinality = number of tuples

Domain = allowed atomic values

CONSTRAINTS

Domain valid values

Key uniqueness

Entity integrity PK cannot be NULL

Referential integrity FK matches referenced key or is permitted NULL

RELATIONAL ALGEBRA

σ selection rows

π projection columns

union either/both

- difference in first, not second

$\times$ Cartesian product all combinations

ρ rename new relation/attribute names

join related combinations

$\div$ division "for all" queries

CALCULUS

TRC: $\{t \mid P(t)\}$ tuple variables

DRC: $\{ \langle x_1, \dots, x_n \rangle \mid P(\dots) \}$ domain variables

Safe expression finite result from active domain

SQL ORDER

FROM WHERE GROUP BY HAVING SELECT DISTINCT ORDER BY

JOINS

INNER matching rows

LEFT all left rows

RIGHT all right rows

FULL all rows from both

CROSS Cartesian product

SELF table joined to itself

NULL

Use IS NULL, not = NULL

COUNT(*) counts rows; most aggregates ignore NULL

SUBQUERIES

IN membership

EXISTS at least one row

ANY comparison true for at least one

ALL comparison true for every value

Double NOT EXISTS relational division

VIEW = stored query/virtual table

MATERIALIZED VIEW = stored query result

TRIGGER = automatic event-condition-action logic

GRANT gives privilege; REVOKE removes privilege

COMMIT saves; ROLLBACK cancels; SAVEPOINT marks partial rollback point

Final Exam Tip

For every relational algebra or SQL answer, first write the participating schemas and identify their keys. Then solve the query in stages—filter, join, group and project. This prevents missing join conditions and unnecessary attributes.